ЧУПО «Высшая школа предпринимательства»

Разработка базы данных для конструктора ПВЗ

Воробьёв Роман Алексеевич
3-й курс • 09.02.07 «Информационные системы и программирование»

Преподаватель: к.ф.н. Ткачев П.С.

ЦЕЛЬ И ЗАДАЧИ

ЦЕЛЬ: Разработать реляционную БД для мультитенантной платформы-конструктора ПВЗ, отражающую ключевые бизнес-процессы электронной торговли.

01 Анализ бизнес-процессов

02 Требования к БД

03 Выбор СУБД

04 ER-диаграмма

05 Физическая структура

06 Реализация & тестирование

ПРЕДМЕТНАЯ ОБЛАСТЬ

Что такое конструктор ПВЗ?

SaaS-платформа с мультитенантной архитектурой: один экземпляр системы обслуживает множество независимых магазинов-арендаторов, логически изолированных друг от друга.

Суперадминистратор
Управляет платформой

Арендатор (ПВЗ)
Независимый магазин с доменом

Менеджер ПВЗ
Обработка заказов

Клиент
Покупатель платформы

20–25%

ежегодный рост
e-commerce в России
(АКИТ, 2025)

14
таблиц БД

SaaS + Multi-tenant

архитектура: один код - тысячи магазинов

MySQL

выбранная СУБД

КЛЮЧЕВЫЕ БИЗНЕС-ПРОЦЕССЫ ПЛАТФОРМЫ

tenant

Управление арендаторами

Регистрация ПВЗ-магазинов, домен, адрес, режим работы, статус активности

auth_user

Управление пользователями

Роли: customer / manager / admin. Профили, разграничение прав доступа

product

Каталог товаров

Мультитенантный каталог: бренды, категории, товары, остатки, JSON-атрибуты

cart

Корзина покупателя

Добавление товаров, изменение количества, привязка к сессии / пользователю

order

Жизненный цикл заказа

pending → confirmed → shipped → delivered → completed / cancelled

payment

Обработка платежей

Разные методы оплаты, статус транзакции, время оплаты, привязка к заказу

site_config

Конструктор сайта

Site Builder: создание страниц drag-and-drop, хранение конфигурации в JSON

notification

Уведомления и мониторинг

Системные уведомления, отслеживание посылок (просрочка >7 дней на ПВЗ)

ТРЕБОВАНИЯ К БАЗЕ ДАННЫХ

Функциональные требования

1. **Хранение данных**
   Тенанты, пользователи, товары, заказы, платежи, корзины, уведомления
2. **Изоляция данных**
   Каждая запись привязана к tenant_id - логическая изоляция арендаторов
3. **Жизненный цикл заказа**
   Полная цепочка статусов со сменой состояний и откатом при отмене
4. **Ссылочная целостность**
   FK, NOT NULL, UNIQUE - автоматический контроль консистентности
5. **JSON-поля**
   База данных должна поддерживать хранение JSON-полей

Нефункциональные требования

1. **Реляционная модель**
   Данные в таблицах, связи по типу «один ко многим»
2. **Нормализация 3NF**
   Нет транзитивных зависимостей, нет избыточности данных
3. **Транзакционная безопасность**
   Поддержка ACID, откат при ошибках (ROLLBACK)
4. **Гибкость системы**
   База данных должна поддерживать большое количество одновременных запросов
5. **Масштабируемость**
   Индексы на FK

ПОЧЕМУ MySQL?

MySQL 8.0

выбранная СУБД

Нативная работа с Django ORM

Полная совместимость без доп. настроек

Бесплатная лицензия GPL

Открытый исходный код, без затрат

Триггеры, процедуры, функции

Все нужные инструменты из коробки

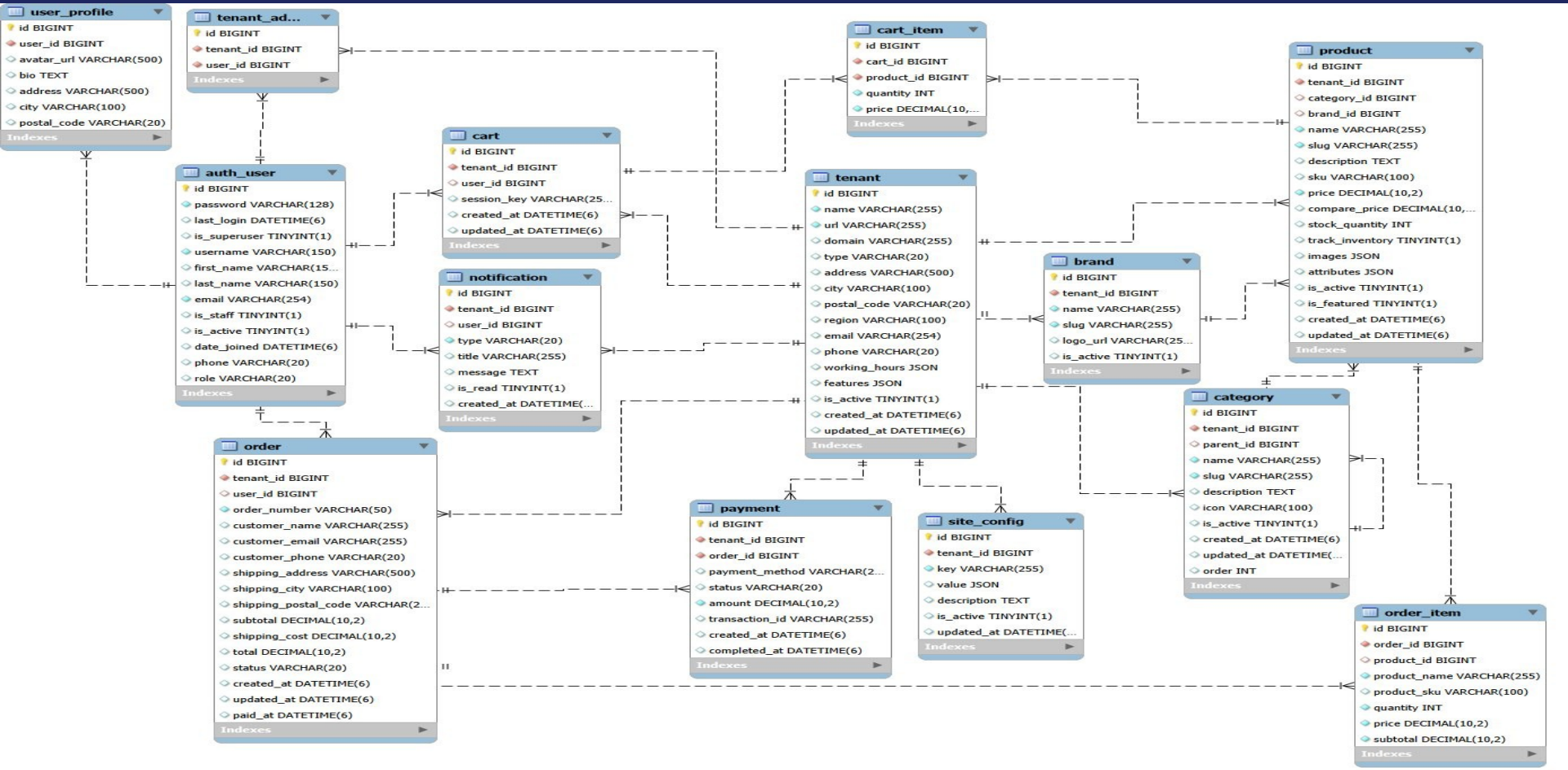
Знакомая технология

Пройден курс МДК.11.01 и ОП 08 по MySQL

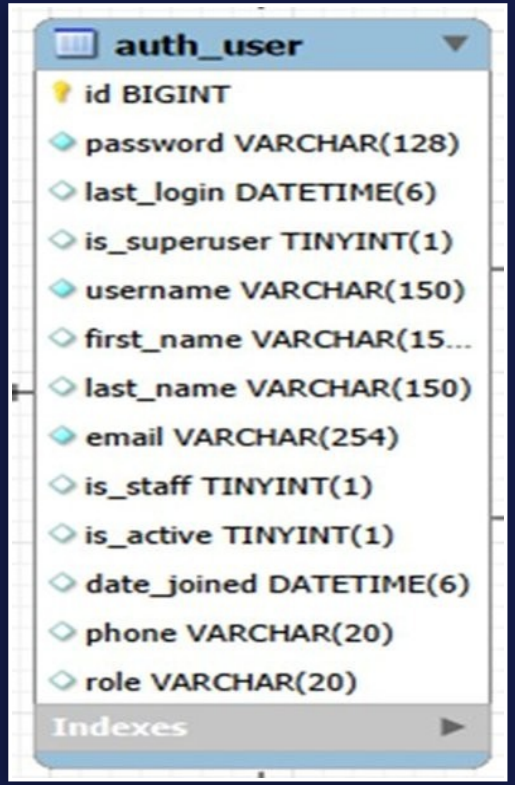
MySQL — оптимальный выбор: бесплатно, знакомо, полностью совместимо с проектом

ER-ДИАГРАММА БАЗЫ ДАННЫХ

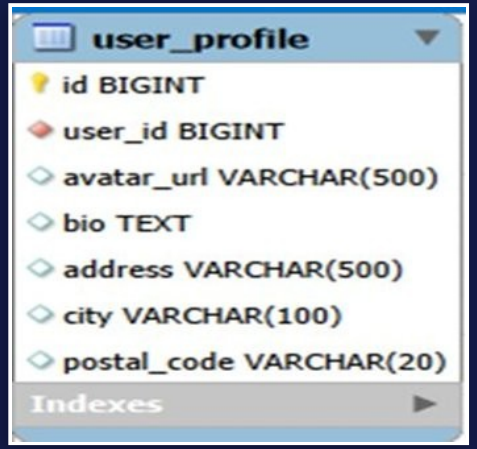
MySQL Workbench • 14 таблиц



СТРУКТУРА ТАБЛИЦ: ПОЛЬЗОВАТЕЛИ И АРЕНДАТОРЫ



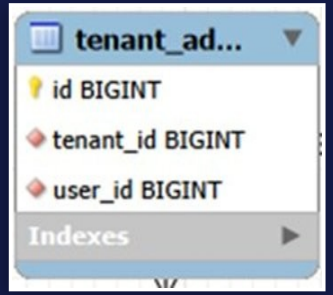
Пользователи системы



Профили

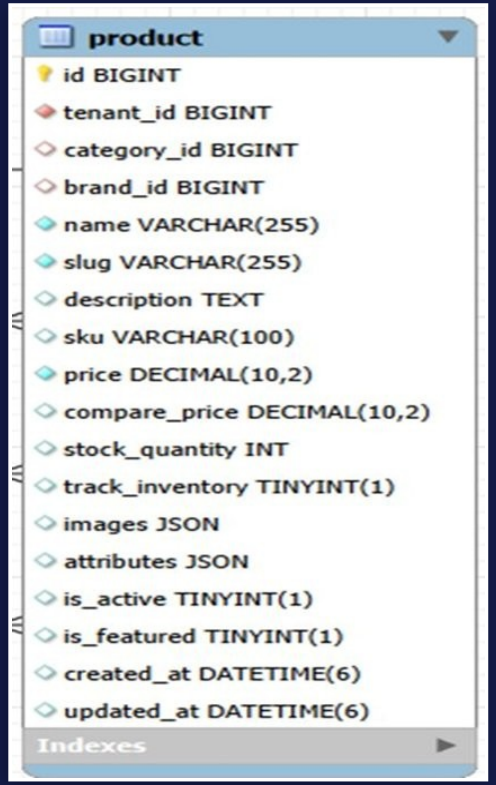
💡 user_profile расширяет auth_user через FK user_id (1:1). Арендаторы хранят рабочие часы и функции в JSON-полях.

Арендаторы

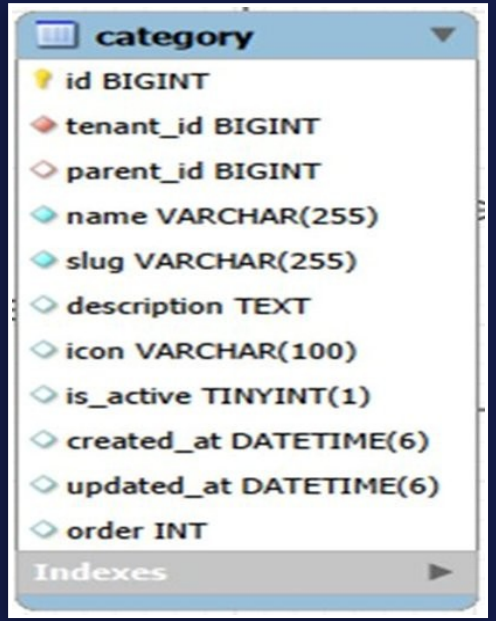


Связь ПВЗ и пользователя

СТРУКТУРА ТАБЛИЦ: КАТАЛОГ ТОВАРОВ



Товары



Категория

Брэнд

Конфигурация сайта

product содержит JSON-поля images и attributes • category.parent_id → self-reference (иерархия) • Каждая запись привязана к арендатору через tenant_id

СТРУКТУРА ТАБЛИЦ: КОРЗИНА, ЗАКАЗЫ, ПЛАТЕЖИ

cart
- id BIGINT
- tenant_id BIGINT
- user_id BIGINT
- session_key VARCHAR(255)
- created_at DATETIME(6)
- updated_at DATETIME(6)

Indexes

Корзина

cart_item
- id BIGINT
- cart_id BIGINT
- product_id BIGINT
- quantity INT
- price DECIMAL(10,2)

Indexes

Связь товаров с корзиной

order
- id BIGINT
- tenant_id BIGINT
- user_id BIGINT
- order_number VARCHAR(50)
- customer_name VARCHAR(254)
- customer_email VARCHAR(254)
- customer_phone VARCHAR(20)
- shipping_address VARCHAR(500)
- shipping_city VARCHAR(100)
- shipping_postal_code VARCHAR(20)
- subtotal DECIMAL(10,2)
- shipping_cost DECIMAL(10,2)
- total DECIMAL(10,2)
- status VARCHAR(20)
- created_at DATETIME(6)
- updated_at DATETIME(6)
- paid_at DATETIME(6)

Indexes

Заказы

order_item
- id BIGINT
- order_id BIGINT
- product_id BIGINT
- product_name VARCHAR(255)
- product_sku VARCHAR(100)
- quantity INT
- price DECIMAL(10,2)
- subtotal DECIMAL(10,2)

Indexes

Товары в заказе

payment
- id BIGINT
- tenant_id BIGINT
- order_id BIGINT
- payment_method VARCHAR(20)
- status VARCHAR(20)
- amount DECIMAL(10,2)
- transaction_id VARCHAR(255)
- created_at DATETIME(6)
- completed_at DATETIME(6)

Indexes

Оплата

notification
- id BIGINT
- tenant_id BIGINT
- user_id BIGINT
- type VARCHAR(20)
- title VARCHAR(255)
- message TEXT
- is_read TINYINT(1)
- created_at DATETIME(6)

Indexes

Уведомления

БИЗНЕС-ЛОГИКА СРЕДСТВАМИ MySQL

Триггер: trg_before_product_update

Тип: BEFORE UPDATE на таблице product

Запрещает отрицательный остаток товара (stock_quantity < 0)

→ Последняя линия защиты, работает даже при прямом SQL-запросе

Процедура: cancel_order(id, reason)

Проверяет: нельзя отменить выполненный или уже отменённый заказ

В транзакции: меняет статус → отменяет платёж → создаёт уведомление

→ Атомарность: всё или ничего. Полный откат при ошибке.

Функция: fn_calc_shipping(tenant_id, sum)

Читает порог бесплатной доставки из site_config арендатора

Если сумма ≥ порогу → возвращает 0.00, иначе → 350 руб.

→ Каждый арендатор настраивает порог в панели без изменения кода

Представления & Роли

Views: active_products, orders_with_details, overdue_parcels_at_pvz

Роли: pvz_manager (SELECT/INSERT/UPDATE), pvz_analyst (только SELECT)

→ Изоляция: каждый арендатор видит только свои данные (tenant_id)

РЕЗУЛЬТАТЫ ПРОЕКТА

- Проанализированы бизнес-процессы мультитенантной платформы ПВЗ
- Обоснован выбор СУБД MySQL на основе сравнительного анализа
- Реализованы триггеры, хранимые процедуры, функции, представления, роли
- БД интегрирована в реально работающий продукт Django Marketplace
- Сформулированы функциональные и нефункциональные требования к БД
- Разработана ER-диаграмма, 14 таблиц, нормализация до 3NF
- Выполнено тестирование всех механизмов на реальных тестовых данных

Спасибо за внимание!

https://github.com/Bibuk/CourseWork.git